Comparative Analysis:

Intent-Driven BIM Generation Approaches

Working Paper — For Peer Review Circulation

Date: 2026-02-19 | Version: 2.0 | Status: Draft

Supplements: Oon, R. D. (2026). The BIM Intent Compiler. Working paper.

Abstract

This paper surveys the emerging landscape of intent-driven Building Information Model generation, comparing five architectural approaches: LLM-mediated API call generation, parametric function composition, rule-based generative layout, AI-assisted classification, and metadata-driven compilation. We locate our own system — a deterministic DSL-to-IFC compiler validated against three reference buildings spanning three construction traditions — within this landscape, and subject it to the same critical scrutiny applied to competing approaches. We report current system outputs with annotated visual evidence (Figures 1–4), trace observable defects to their root causes in the metadata layer, and identify the open problems shared across all approaches. The goal is accurate characterisation of the field's state, not advocacy for any single method.

1. The Problem Space

The AEC industry faces a persistent productivity challenge: translating design intent into construction-ready Building Information Models remains labour-intensive, error-prone, and inaccessible to practitioners without specialised BIM authoring training. The Industry Foundation Classes standard (ISO 16739) provides a rich semantic framework for building data exchange, but IFC file creation has historically required either manual modelling in commercial tools or bespoke programming against IFC toolkits.

Several research groups and commercial ventures are now exploring automated or semi-automated approaches to this problem. This document surveys the emerging landscape, locates our own work within it, and identifies the open problems that remain unsolved across all approaches.

We organise the discussion around a central question: *given a declarative specification of building intent, what are the formal properties of the generated IFC output, and what guarantees can be offered about its correctness?*

2. Taxonomy of Approaches

Current work can be classified along two axes: the input modality (how intent is expressed) and the generation mechanism (how IFC output is produced). This taxonomy is necessarily simplified; real systems often span multiple categories. The categories are intended to clarify architectural differences in how correctness is (or is not) guaranteed.

Approach	Input Modality	Generation Mechanism	Representative Systems
LLM-mediated API calls	Natural language	LLM generates imperative code invoking BIM tool APIs	Text2BIM (Yue et al., 2025)
Parametric function composition	Parameter sets / text	Pre-authored functions composed via cloud execution	Hypar (Keough & Hauck)
Rule-based generative layout	Site constraints + programme	Constraint solvers + heuristic placers	TestFit, Finch, Spacio
AI-assisted classification	Existing geometry	ML classifiers assign IFC semantics	BricsCAD BIMIFY, Aurivus
Metadata-driven compilation	Domain-specific language	Deterministic DAG expansion from metadata	BIM Intent Compiler (this work)

3. Detailed Comparison

3.1 LLM-Mediated Approaches: Text2BIM and BIM-GPT

Text2BIM (Yue et al., 2025; arXiv:2408.08054) presents a multi-agent LLM framework that transforms natural language building descriptions into IFC models via the Vectorworks BIM authoring API. The system employs a Programmer agent (generating API calls), a Reviewer agent (interpreting rule-check results), and a domain-specific model checker that evaluates output against geometric analysis, collision detection, and information verification criteria. Results are exported in BIM Collaboration Format (BCF) for iterative refinement. BIM-GPT (Zheng et al., 2023) explores natural language querying of existing BIM models, establishing the broader pattern of LLM-BIM interaction.

Strengths. The natural language interface is the most accessible input modality. Users need no DSL training, no programming ability, and no knowledge of IFC semantics. The multi-agent architecture is a genuine contribution — the separation of generation, checking, and review mirrors good software engineering practice. The BCF-based feedback loop is an elegant use of existing standards.

Open problems. The fundamental challenge is non-determinism. Given the same textual input, the LLM may generate different API call sequences across runs, potentially producing different models. This is a structural characteristic of autoregressive language models, not a deficiency of the Text2BIM implementation specifically. The practical consequence is that reproducibility requires either fixing the random seed (which constrains utility) or accepting that output may vary. The second challenge is verification scope: the model checker validates geometric properties and collisions, which are necessary but not sufficient conditions for construction-readiness. The checker operates post-hoc; it cannot provide guarantees about properties not explicitly encoded. The third challenge is tool dependency: generating code for the Vectorworks API ties the system to a specific commercial platform.

3.2 Parametric Function Composition: Hypar

Hypar, founded by Ian Keough (creator of Dynamo) and Anthony Hauck (former head of Revit product development), is a cloud platform for generative building design. It defines BIM elements using JSON schema, executes parametric functions as cloud microservices, and exports to IFC and Revit formats. The “Text-to-BIM” capability, demonstrated publicly from 2023, wires the OpenAI API to Hypar’s function execution engine, where natural language descriptions are interpreted and mapped to pre-authored parametric functions. Hypar secured $5.5M Series A funding in June 2023 and reports over 2,000 users. Recent work with DPR Construction on automated drywall layout demonstrates extension toward fabrication-level detail.

Strengths. Hypar’s open function ecosystem allows community-contributed generators, creating a marketplace of building generation algorithms. The platform handles execution infrastructure (cloud compute, 3D viewer, collaboration), freeing developers to focus on design logic. The Revit integration supports round-trip workflows. The function marketplace model incentivises contributions from the broader AEC development community.

Open problems. Output quality is bounded by the parametric functions available. A building type without a corresponding function cannot be generated. Construction knowledge is embedded implicitly in each function’s source code (C# or Python), not in a queryable data structure. Auditing what building code rules a particular function enforces requires reading its source. There is no separation between design knowledge and generation algorithm — they are co-mingled in the function implementation. The text-to-BIM capability inherits LLM non-determinism.

3.3 Rule-Based Generative Layout

TestFit, Finch, and Spacio focus on early-phase design feasibility. Given site constraints and a building programme (unit mix, parking requirements, floor-area ratios), they generate candidate building massings and floor plans. TestFit targets real-estate feasibility with real-time cost and constructability feedback, reporting approximately 4× faster site planning and 3× more design variants compared to manual workflows. Finch integrates with Revit and Rhinoceros for apartment layout optimisation. Spacio generates building massings from plot data with contextual analysis.

Strengths. These tools solve a genuine industry pain point — early-phase design exploration, producing useful results in minutes rather than weeks. They support iterative refinement through parameter adjustment and several offer IFC export for downstream use.

Open problems. These systems typically operate at LOD 100–200 (conceptual massing and approximate geometry). They do not generate construction-level detail (LOD 400 components, assembly hierarchies, MEP systems). The output is a starting point for detailed design, not a construction-ready model. This is by design — the tools target a different phase — but it means they do not address the intent-to-construction-document pipeline.

3.4 AI-Assisted Classification

BricsCAD's BIMIFY analyses solid geometry and automatically classifies building elements according to IFC standards, reporting approximately 95% accuracy on standard elements. The PROPAGATE feature detects similar junction conditions across a model and replicates details automatically. Scan-to-BIM systems such as Aurivus use ML algorithms to identify architectural and MEP elements in point cloud data.

Strengths. These tools augment existing workflows rather than replacing them. They accept the reality that most buildings are designed in general-purpose CAD tools and add BIM semantics after the fact. The classification accuracy is practical for production use.

Open problems. Classification is not generation. These tools require geometry to already exist; they add semantic labels and relationships. They cannot produce a building from intent. The residual 5% classification error rate, while acceptable for assisted workflows, means automated downstream processing (code checking, quantity takeoff) requires human verification of classification results.

4. The BIM Intent Compiler: Position and Limitations

Our system takes a different architectural position from all of the above: deterministic compilation from a domain-specific language through metadata-driven DAG expansion, with mathematical verification of output correctness. This section describes the approach and then subjects it to the same critical scrutiny applied to competing approaches in Section 3.

4.1 Architecture Overview

The system operates in two phases that are architecturally distinct but share a common database intermediary:

Phase 1: Extraction. Source IFC files are parsed by IfcOpenShell and decomposed into a FederatedModel DB schema (SQLite). Tessellated geometry is stored as vertex/face arrays; spatial data as bounding boxes; semantic data as IFC class tags and property sets. This extraction is lossless at LOD 400 and produces a SQL-queryable representation of the building. The extraction pipeline is contributed to the IfcOpenShell open-source project (github.com/red1oon/IfcOpenShell). The DB is the prerequisite container for all subsequent work.

Phase 2: Compilation. A DSL expressing building intent (20–80 lines) drives a Java resolver engine that expands declarations into positioned IFC elements through a five-stage DAG pipeline (Parse → Resolve → Compile → Bind → Write). The compiler reads from 55 Application Dictionary tables in the same SQLite database. The Bind stage enforces dimensional coherence between placement (bounding boxes) and mesh (tessellated geometry) through a proof-carrying type called BoundElement. The DAG compiler is a separate project (github.com/red1oon/BIMCompiler).

The two phases serve different roles in the Rosetta Stone methodology. Extraction produces the ground truth reference from existing IFC files. Compilation produces new output from declarative intent. The X-ray spatial comparison validates compilation output against extraction output. For generative buildings (no source IFC), only Phase 2

applies — the compiler produces output from metadata alone, with no extraction reference to validate against. Figures 1–4 show outputs from both phases.

4.2 Current Output: Visual Evidence

We present the system's current output without selection bias. Figures 1–3 show compiler output (Phase 2) for two Rosetta Stone buildings and one generative building. Figure 4 shows extraction output (Phase 1) for the third Rosetta Stone. Each Blender viewport image is annotated; orange highlighting in the outliner indicates elements with residual issues.

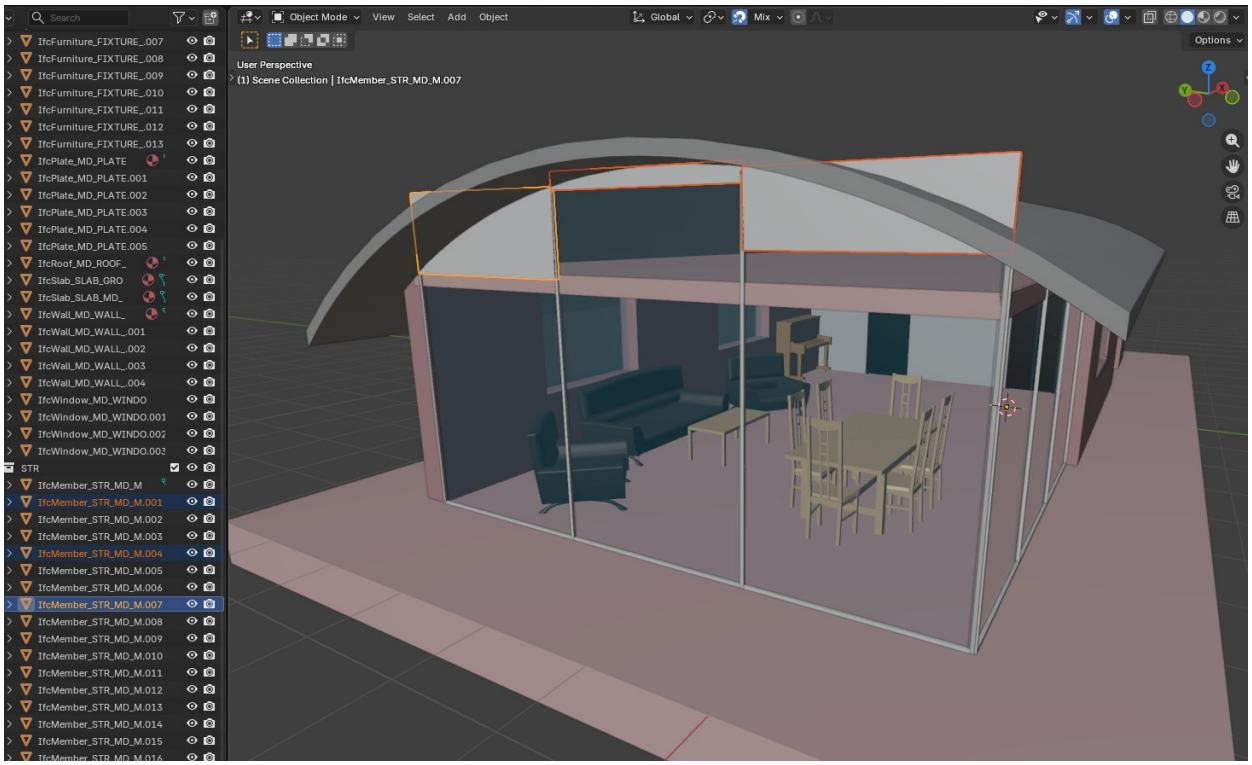

Figure 1. *SampleHouse (UK residential, 55 elements) — Rosetta Stone 1. Compiled from 55 relational rules. Note orange-highlighted IfcMember elements (.001, .004, .007): curtain wall mullions and glass panels extend above the curved roof profile. This is the roof-glass intersection defect traced to missing clipping rules in the metadata layer (see Table 3).*

SampleHouse *(Figure 1)* is a UK detached residential building with a distinctive curved barrel-vault roof and full-height curtain wall system. The compiler achieves 100% spatial recall (55/55 elements) against the reference IFC. The remaining visual defect is visible in the figure: three structural mullions (IfcMember, aluminium) and associated glass

panels (IfcPlate, transparency 0.9) extend through the curved roof envelope. The elements are spatially correct at the bounding-box level — the X-ray confirms their centroid positions match the reference — but their height extends to full storey height without being clipped to the roof curve. This is traced to a missing clipping rule in the ad_element_rule table: the mullion height_mm is set to the full storey height rather than being parameterised against the roof profile at each X-position.

Figure 2. *Duplex (US residential, 1085 elements) — Rosetta Stone 2. Compiled from 1085 relational rules. Note orange-highlighted windows: stairwell windows on the left facade show exaggerated depth (shelf-like protrusion) caused by depth_mm storing room dimension rather than wall thickness. Small window at far left also exhibits the depth anomaly.*

Duplex *(Figure 2)* is a US two-storey semi-detached residential building with 1,085 elements including MEP piping across 9 disciplines. The compiler achieves 100% spatial recall. The orange-highlighted defect is on the left facade: stairwell windows (IfcWindow, three stacked vertically) exhibit exaggerated depth, giving them a shelf-like

protrusion from the wall face. A narrow window at far left shows the same anomaly. This is the documented “depth_mm semantic” gap: the window’s depth_mm column in ad_element_rule stores the room dimension perpendicular to the wall rather than the wall construction thickness. The data fix is to recompute depth_mm from the host wall’s ad_wall_type.total_mm value (EXTERIOR_BRICK = 417mm in this case). The green flat roof visible at top is correct for this building type.

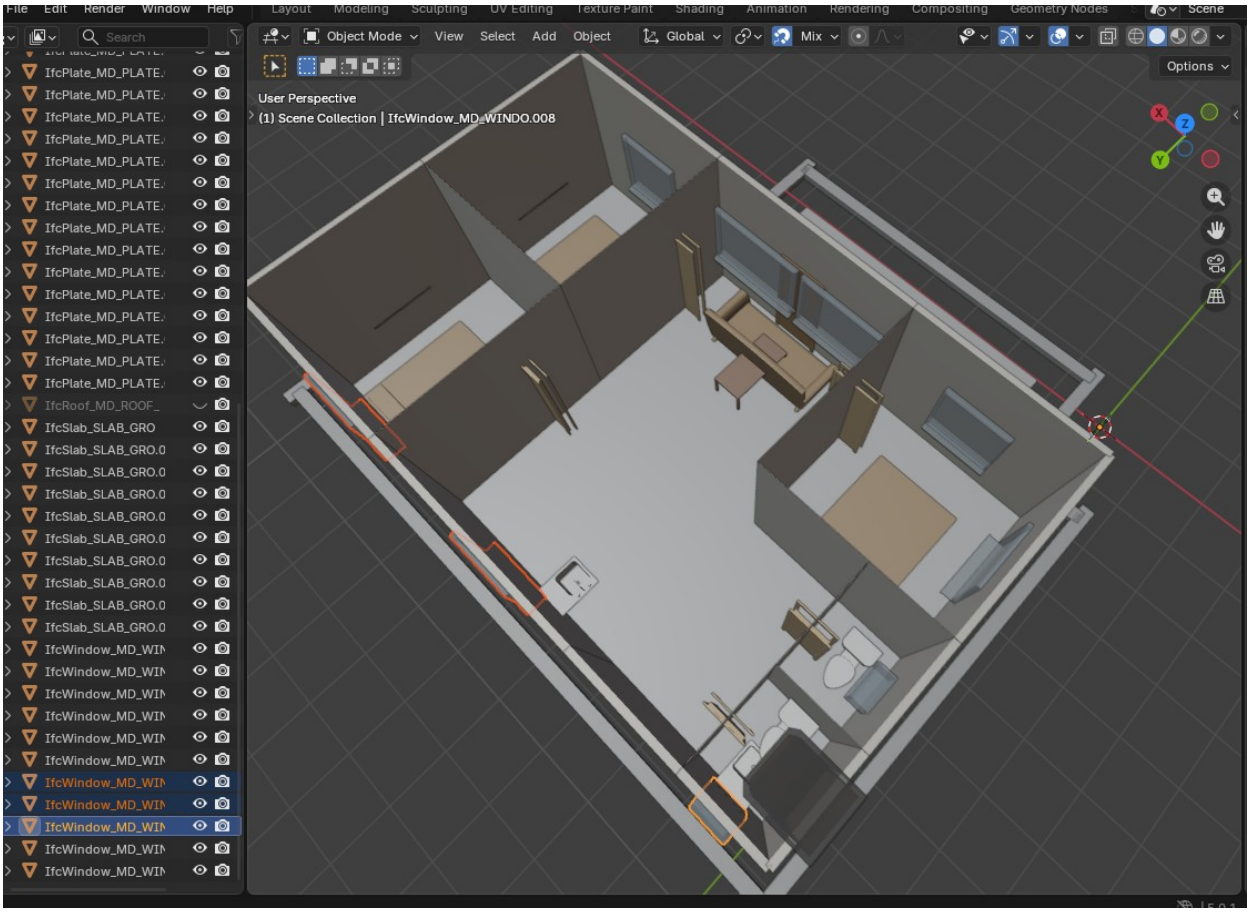

Figure 3. *CitizenHome / TB-LKTN (Malaysian affordable housing, 58 elements) — Generative building, no IFC reference. Top-down cutaway view. Multiple visible defects: flat roof (should be 25° pitched gable), doors not rotated to face corridor, furniture blocking main entrance, toilet bowl orientation incorrect, kitchen and living areas sparsely furnished, orange-highlighted windows with orientation issues.*

CitizenHome *(Figure 3)* is a Malaysian affordable housing unit (Taman Baru Lembah Keramat Tambahan Nasional) — the system’s first fully generative building with no IFC reference file. All 58 elements are compiled from relational rules and metadata. This building exposes the gap between spatial correctness (all elements are in their correct

rooms at computed grid coordinates) and visual correctness. Table 2 catalogues seven observable defects and traces each to a specific metadata gap. The building is deliberately presented without remediation to illustrate the current distance between "spatially valid" and "construction-ready."

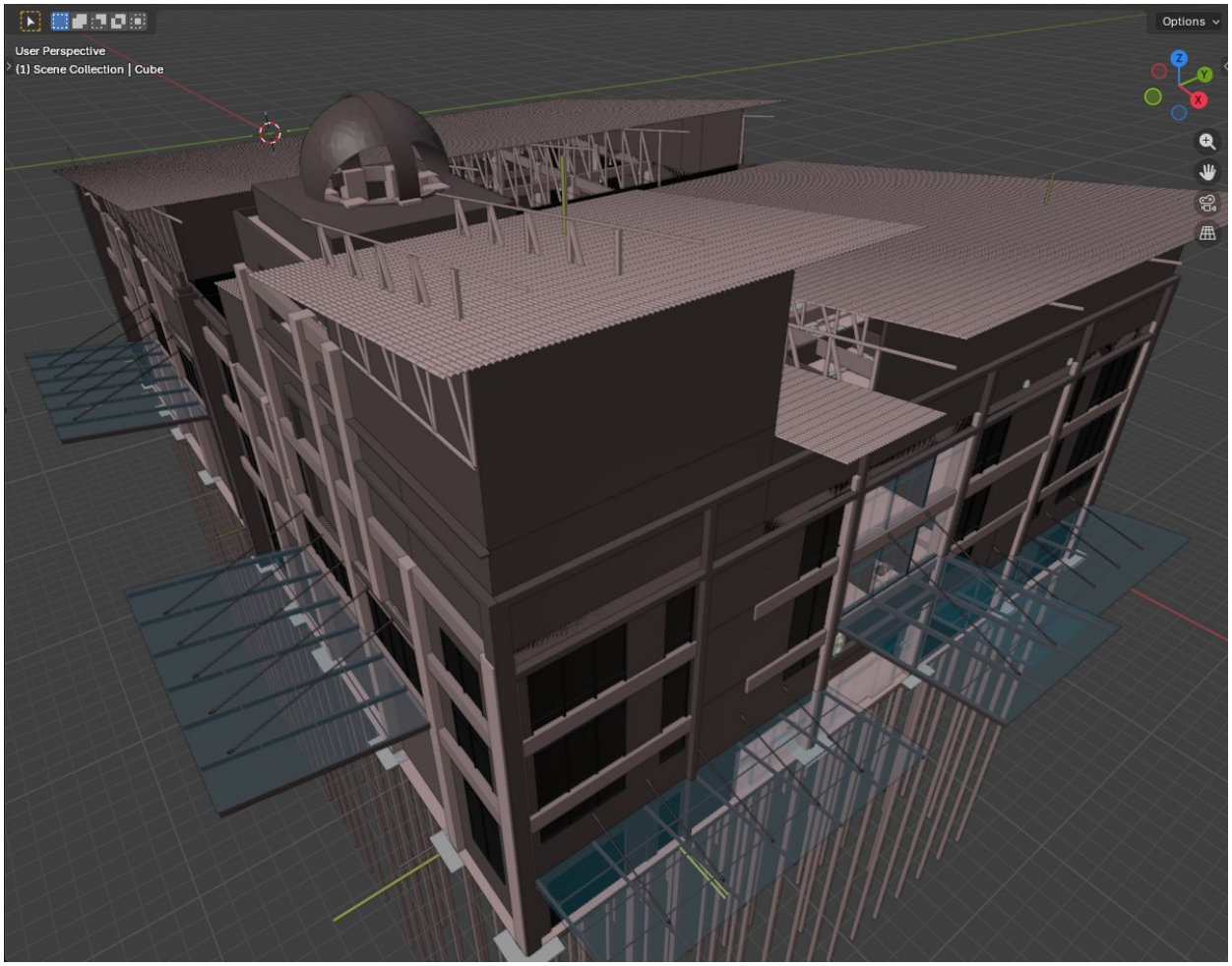

Figure 4. *Terminal (Malaysian institutional, 51,088 elements) — Rosetta Stone 3 (extraction only). This image shows the extraction pipeline output, partial recompilation. The source IFC file was extracted into the FederatedModel DB schema (SQLite), then rendered in Blender via the Bonsai addon. Visible: multi-storey concrete frame, roof trusses, dome, glass curtain walls with correct transparency, canopy structures. 51,088 elements rendered from SQL queries — a task that would crash most IFC viewers if loaded directly from the source file.*

Terminal *(Figure 4)* is a Malaysian institutional building (51,088 LOD 400 elements across 9 disciplines including structural, architectural, mechanical, electrical, plumbing, fire protection, and drainage). **This figure shows extraction output, with staged recompilation.** The distinction is important: the source IFC was parsed by IfcOpenShell,

decomposed into the FederatedModel DB schema (SQLite with tessellated geometry stored as vertex/face arrays), and rendered in Blender via the Bonsai federation addon. This extraction pipeline is the prerequisite stage for all compiler work — the DB is the container from which the DAG compiler reads its ground truth. The Terminal has not yet fully applied through the relational metadata pipeline; its 51,088 elements currently use FLAT mode (pre-extracted coordinates in ad_element_placement). Recompilation through the relational pipeline is deferred until the ad_room_boundary and ad_wall_face tables are populated for this building — a substantial metadata authoring task given the building's complexity.

The Terminal image demonstrates a separate but essential contribution: the FederatedModel DB schema itself, contributed to the IfcOpenShell open-source project. By extracting IFC data into SQLite, the system enables SQL-queryable access to building geometry that would otherwise require specialised IFC parsing libraries. This extraction is lossless at LOD 400 — every vertex and face is preserved — and the resulting database can be rendered in Blender's viewport where direct IFC loading of 51,000+ elements would typically exhaust available memory.

Table 2. *CitizenHome (TB-LKTN) defect catalog with data-layer root cause tracing*

Defect	Observable Symptom	Root Cause in Data Layer	Proposed Fix
D1. Flat roof	IfcRoof emitted as flat slab; should be 25° pitched gable	IfcRoof_1 uses ENVELOPE rule with library mesh (8V, 12F). Mesh encodes a gable profile but the roof pitch parameter is not applied during placement.	Add pitch_degrees column to ad_element_rule or create ad_roof_profile table; modify MeshBinder to apply rotation transform.
D2. Doors not rotated	All 6 doors face canonical orientation, not corridor	ad_element_rule has no orientation/rotation column. FRACTION placement computes position along wall but does not resolve which side the door swings toward.	Add orientation_deg or facing_rule column to ad_element_rule; RelationalResolver applies rotation based on adjacent_room in ad_wall_face.
D3. Furniture blocks main door	Sofa (FE_9) placed at centre of common room, coinciding with main door position	FE_9 uses position_value=0.5 (FRACTION) in common room. IfcDoor_1 also at FRACTION 0.5 on WALL_common_SOUTH. Same fraction = spatial collision.	Add clearance_rule or door_avoidance_mm to ad_bom_child_param; FixturePlacer offsets furniture from door zones.
D4. Toilet bowl facing	WC in tandas faces wrong direction; appears as right-	FE_5 (tandas, WC_Seating, FRACTION 0.5) has no rotation data. Library	Add rotation_deg to furniture placement in ad_element_rule; derive from host room wall adjacency.

	facing rather than wall-facing	mesh has canonical orientation that may not align with room layout.	
D5. Kitchen sparse	Only sink visible in kitchen zone; no stove, refrigerator, or counter	Only FE_1 (Kitchen_Sink_Aluminium) targets the common room kitchen area. No BOM rules in ad_bom for kitchen assembly.	Add ad_bom entries for KITCHEN room_type: stove, fridge, counter, upper cabinets. Each becomes a BOM child with spatial offset.
D6. Living room bare	Only sofa + coffee table in 22.94 m^2 living area	FE_9 (Sofa_Fabric) and FE_10 (Coffee_Table_Wood) are the only common-room furniture. No TV unit, dining table, or side tables in ad_element_rule.	Expand ad_bom for LIVING room_type with typical Malaysian affordable housing furnishing schedule.
D7. Window orientation	South-facing windows highlighted orange; east/west windows may have width/depth swap	Windows 8–11 on EAST/WEST walls note “swapped width/depth for orientation” in the data audit. The 100mm/1200mm values may be transposed.	Validate width_mm vs depth_mm convention for wall-perpendicular vs wall-parallel axes; correct in ad_element_rule.

Table 3. *Rosetta Stone residual defects (SampleHouse and Duplex) with data-layer tracing*

Building	Defect	Affected Elements	Data Root Cause	Fix
Sample-House	Roof-glass penetration	IfcMember .001, .004, .007; IfcPlate .000–.005	ad_element_rule: mullion/panel height_mm = full storey (2700mm). No clipping rule against IfcRoof envelope.	Add max_z_rule = ROOF_ENVELOPE to affected IfcMember and IfcPlate rows, or add ad_clipping_rule table.
Duplex	Window depth protrusion	IfcWindow .017 + stairwell stack	ad_element_rule: depth_mm stores room perpendicular dimension, not ad_wall_type.total_mm for host wall.	Recompute: depth_mm = ad_wall_type.total_mm WHERE wall_type_id = host wall's type (EXTERIOR_BRICK → 417mm).

4.3 Formal Properties

Determinism. The same DSL input always produces the same output. The SpatialDigest (SHA-256 hash of all element bounding boxes at 1mm precision) serves as a regression test — any change to any element's position or dimensions changes the digest. This property is enforced by deterministic emission order and the absence of stochastic elements in the pipeline.

Proof-carrying output. The BoundElement record type enforces at construction time that every element's mesh geometry fits within its placement bounding box, with scale factors constrained to [0.3, 3.0] per axis and 2mm tolerance. An element that violates this contract is explicitly marked as degraded with the violation recorded in the witness output. The PlacementProver audits 14 spatial and topological properties across 5 tiers of increasing scope. These are structural guarantees, not post-hoc checks; the proof-carrying type makes it impossible to write unverified geometry through the primary code path.

Metadata-data separation. All construction knowledge — wall thicknesses, opening families, fixture clearances, BOM recipes, material properties — resides in queryable SQLite tables (55 Application Dictionary tables), not in procedural code. Adding a new building variant requires SQL INSERT statements, not Java code changes. This separation means construction knowledge is auditable independently of the generation algorithm. It also means that every defect visible in Figures 1–4 traces to a specific table and column (Tables 2–3), not to algorithmic error.

4.4 Open Problems and Honest Limitations

4.4.1 Vocabulary bootstrapping (shared problem). Every approach requires initial construction knowledge: Text2BIM relies on LLM training data plus tool-embedded constraints; Hypar relies on developer-authored functions; our system relies on populated metadata tables. The “extract, don’t imagine” principle means we require validated reference IFC models before supporting a new building type. The compound enrichment model reduces marginal effort over time, but initial investment per building tradition remains significant.

4.4.2 Geometry fidelity gap (specific to this work). The spatial X-ray compares axis-aligned bounding boxes, not mesh vertices. Rotated elements, L-shaped walls, and curved geometry are approximated. The BoundElement proof-carrying type bridges the dimensional gap but does not eliminate it: elements requiring mesh scaling beyond 1.5× on any axis produce visually noticeable distortion. True resolution requires per-instance parametric mesh generation or constructive solid geometry from metadata — both future work.

4.4.3 Natural language accessibility (different tradeoff). The DSL requires learning a small grammar. We chose determinism over natural language input because determinism is a prerequisite for proof-carrying output. A GUI editor over the metadata tables would mitigate the accessibility barrier without sacrificing determinism, but does not yet exist.

4.4.4 Hardcoded residuals. An audit identified 18 of 33 hardcoded values remaining unwired to the metadata layer: 6 in the architectural discipline (beam span-to-depth ratios, stair widths, corridor widths) and 6 requiring new metadata tables for MEP and structural disciplines. These represent areas where the “all data, nothing hardcoded” principle has not yet been fully realised.

4.4.5 Scale limitations (unknown). Validated on 55 to 51,088 elements. Empirical validation at the scale of large commercial buildings (500,000+ elements) or campus-scale models is needed.

4.4.6 Single-material limitation. Each element carries one material_rgba value. Real building elements — particularly windows, doors, and curtain walls — are composite (frame, glazing, hardware). IFC represents this through IfcMaterialConstituentSet, but our pipeline collapses to a single value. This produces visual artefacts that do not affect spatial correctness but degrade visual fidelity.

5. The Verification Problem: A Cross-Cutting Concern

Across all approaches, the most significant open problem is verification of generated output. Each approach defines "correct" differently:

System	Verification Method	What It Proves	What It Does Not Prove
Text2BIM	Rule-based checker + BCF feedback	Geometry valid, no collisions, info present	Reproducibility, system connectivity, code compliance
Hypar	Visual inspection + Revit validation	Model exports cleanly, looks right	Spatial accuracy vs. reference, determinism
TestFit / Finch	Parameter constraint satisfaction	Programme requirements met	Construction detail, MEP feasibility
BricsCAD BIMIFY	ML classification confidence (≈95%)	Element types correct	Spatial relationships, assembly correctness
BIM Intent Compiler	Rosetta Stone X-ray + proofs	100% spatial match, 14 topological properties	Mesh fidelity, full code compliance, MEP performance

No current system provides complete verification. The verification problem is arguably harder than the generation problem — it requires formalising "what makes a building correct," which touches building codes, engineering standards, constructability constraints, and occupant requirements simultaneously. Our contribution is the Rosetta Stone Methodology: a reproducible, measurable, and honest verification approach using reference IFC files as ground truth. Its limitation is that it requires reference buildings to exist, constraining verification to building types for which references are available.

6. Toward Convergence

Hybrid architectures. Hypar's combination of parametric functions with LLM-mediated selection anticipates a pattern where deterministic generators are orchestrated by stochastic selectors. A metadata-driven compiler could serve as the deterministic backend in such an architecture, with an LLM translating natural language to DSL as a preprocessing step, provided the determinism boundary is clearly maintained.

Standardised verification. The buildingSMART Validation Service and ISO 19650 series point toward industry-standard model checking. As these mature, all generation approaches will need to pass standardised checks, creating a common evaluation framework.

Open IFC ecosystems. The shift toward IFC4 and IFC4.3 (infrastructure) creates interoperability opportunities. A massing from TestFit could be enriched by a metadata-driven compiler, then validated by a standardised checker — each tool contributing its strength to a federated pipeline.

7. Summary

The BIM generation landscape is early-stage and diverse. LLM-mediated approaches offer accessibility; parametric platforms offer developer ecosystems; rule-based tools offer rapid feasibility; classification tools bridge legacy workflows. Metadata-driven compilation offers determinism, proof-carrying output, and auditable separation of construction knowledge from generation algorithm.

No approach has solved the full problem. The most significant shared challenges are vocabulary bootstrapping (how to encode construction knowledge), verification completeness (how to prove output correctness), and the tension between accessibility (natural language input) and formal guarantees (deterministic compilation). The field will benefit from clearly characterising these tradeoffs. The problems are hard enough that multiple complementary strategies will be needed, and the construction industry is

large enough to support diverse solutions — from rapid feasibility studies to construction-ready IFC output.

References

Borrmann, A., König, M., Koch, C., & Beetz, J. (Eds.). (2018). Building Information Modeling: Technology Foundations and Industry Practice. Springer.

BIMForum. (2023). Level of Development (LOD) Specification Part I & Commentary. Version 2023.

ISO. (2024). ISO 16739-1:2024 Industry Foundation Classes (IFC) for data sharing in the construction and facility management industries.

Keough, I. & Hauck, A. (2018–present). Hypar: Generative Building Design Platform. https://hypar.io

Oon, R. D. (2026). The BIM Intent Compiler: A DSL-Driven DAG Compiler for Spatially-Verified IFC Output from Declarative Building Descriptions. Working paper. Source: https://github.com/red1oon/BIMCompiler

Oon, R. D. (2026). FederatedModel DB Schema: SQL-Queryable IFC Extraction for Bonsai/IfcOpenShell. Contributed to IfcOpenShell. Source: https://github.com/red1oon/IfcOpenShell

Robinson, A. (2009). Writing and Script: A Very Short Introduction. Oxford University Press.

Xu, S., Wang, J., Shou, W., Ngo, T., Sadick, A.-M., & Wang, X. (2022). Integrating BIM and AI for Smart Construction Management: Current Status and Future Directions. Archives of Computational Methods in Engineering, 29, 1081–1110.

Yue, D., Chen, Y., Li, Y., et al. (2025). Text2BIM: Generating Building Models Using a Large Language Model-based Multi-Agent Framework. arXiv preprint arXiv:2408.08054v1.

Zheng, Z., Xie, M., et al. (2023). BIM-GPT: a Prompt-Based Virtual Assistant Framework for BIM Information Retrieval. arXiv preprint arXiv:2304.09333.

Appendix A. Current System Metrics

These numbers are provided for reproducibility and honest assessment, not as competitive benchmarks. Different systems optimise for different outcomes; direct numerical comparison across fundamentally different approaches would be misleading.

Metric	Value	Caveat
Rosetta Stone recall (SampleHouse)	100% (55/55)	ARC discipline, bbox-level spatial match
Rosetta Stone recall (Duplex)	100% (1085/1085)	ARC discipline, bbox-level spatial match
Rosetta Stone recall (Terminal)	≈100% (51088/51092)	FLAT mode extraction; relational recompilation deferred
Generative building (CitizenHome)	N/A (58 elements)	No reference — generative output, visual review only
DSL lines per building	20–80	Regardless of output element count (55–51088)
Metadata tables	55	Application Dictionary pattern (iDempiere lineage)
PlacementProver coverage	14 proofs, 5 tiers	Non-blocking audit, not gating
Hardcoded residuals	18 / 33 identified	Active reduction; 12/12 Category A resolved
Construction traditions	3 (UK, US, Malaysian)	Residential + institutional
LOD of output	LOD 400 (library) / parametric	Library = extraction-grade; parametric = box

Appendix B. The Data Layer: How Metadata Drives Compilation

This appendix provides an in-depth exposition of the metadata tables that drive the compiler, using the CitizenHome (TB-LKTN) generative building as a worked example. The purpose is twofold: to demonstrate concretely how declarative metadata produces positioned IFC elements, and to show that every observable defect in Figure 3 traces to a specific table cell rather than to algorithmic error.

All data shown below is queried from a single SQLite database (component_library.db) which serves as the compiler's sole source of truth. The compiler Java code is a resolver: it reads these tables and computes geometry. It does not contain building-specific values.

B.1 System Configuration

Runtime behaviour is governed by key-value configuration rows, not code branches:

```
ad_compiler_config
──────────────────────────────────────────────
config_key          | config_value  | Effect
──────────────────────────────────────────────
placement_mode      | RELATIONAL    | Use rules, not flat coords
closest_fit         | true          | Library mesh closest match
exclude_ifc_class   | IfcOpening    | Skip opening voids
```

Switching to flat (pre-computed) placement requires one SQL UPDATE, no code change. CitizenHome has no flat data — it can only compile in RELATIONAL mode. This is the proof that the system is generative.

B.2 Structural Grid

The building's spatial skeleton is defined by named grid lines in two axes:

Axis	Grid Line	Coordinate (mm)	Meaning
X	A	0	Left exterior

X	B	1300	Wet room strip boundary
X	C	3100	Master bedroom / common boundary
X	D	6800	Common / bedroom 2 boundary
X	E	9900	Right exterior
Y	1	0	Porch front edge
Y	2	2300	Front wall / porch-to-interior
Y	3	5400	Bedroom rear edges
Y	4	7000	Toilet-to-bathroom boundary
Y	5	8500	Rear exterior wall

Building footprint: 9,900 × 8,500 mm = 84.15 m². Grid coordinates are in millimetres, origin at front-left corner.

B.3 Room Layout

Each room is defined by its grid extent, from which the compiler computes absolute coordinates:

Room Name	Type	Grid (X)	Grid (Y)	Computed Size	Area
anjung	PORCH	B–D	1–2	5500 × 2300 mm	12.65 m²
bilik_utama	BEDROOM	A–C	2–3	3100 × 3100 mm	9.61 m²
bilik_2	BEDROOM	D–E	2–3	3100 × 3100 mm	9.61 m²
bilik_3	BEDROOM	D–E	3–5	3100 × 3100 mm	9.61 m²
common	LIVING	C–D	2–5	3700 × 6200 mm	22.94 m²
tandas	TOILET	A–B	3–4	1300 × 1600 mm	2.08 m²
bilik_mandi	BATHROOM	A–B	4–5	1300 × 1500 mm	1.95 m²

The compiler derives room coordinates from the grid: bilik_utama spans grid A–C on X (0–3100mm) and grid 2–3 on Y (2300–5400mm). All downstream element placement references these room boundaries.

B.4 Wall Faces and Adjacency

Each room face declares its wall type and what lies on the other side. This drives both wall geometry and door/window placement:

Room	Face	Wall Type	Exterior?	Adjacent Room
bilik_utama	SOUTH	EXTERIOR_V1	Yes	—
bilik_utama	WEST	EXTERIOR_V1	Yes	—
bilik_utama	EAST	INTERIOR_V2	No	common
bilik_utama	NORTH	INTERIOR_V2	No	tandas
common	SOUTH	EXTERIOR_V1	Yes	— (main entrance)
common	NORTH	EXTERIOR_V1	Yes	—
tandas	WEST	EXTERIOR_V1	Yes	—
bilik_mandi	WEST	EXTERIOR_V1	Yes	—
bilik_mandi	NORTH	EXTERIOR_V1	Yes	—
anjung	*	NONE	N/A	(porch — no walls)

Data gap: EXTERIOR_V1 and INTERIOR_V2 are referenced here but have no matching rows in ad_wall_type. The compiler falls back to 100mm depth for all TB-LKTN walls. The fix is two SQL INSERT statements:

```
INSERT INTO ad_wall_type (wall_type_id, total_mm, description)
VALUES ('EXTERIOR_V1', 150, 'MY Affordable - half-brick plaster');
```

```
INSERT INTO ad_wall_type (wall_type_id, total_mm, description)
VALUES ('INTERIOR_V2', 100, 'MY Affordable - lightweight partition');
```

B.5 Element Placement Rules

Each of the 58 elements has a row in ad_element_rule specifying what to place, where, and from what material. Elements use one of three position rules: BOUNDARY (placed at room edge), FRACTION (fraction along host wall), or ABSOLUTE (world coordinates). Selected entries by IFC class:

Walls (20 IfcPlate)

Element	Host Reference	Position	Width (mm)	Depth (mm)	Material
IfcPlate_1	WALL_bilik_utama_SOUTH	1550.0	3100	100	Teablock_V1_100
IfcPlate_7	WALL_common_SOUTH	4950.0	3700	100	Teablock_V1_100
IfcPlate_12	WALL_bilik_utama_EAST	3100.0	4.0	3100	Teablock_V2_4

Note: Interior walls (Teablock_V2_4) show width_mm=4.0 and depth_mm=3100 — an apparent column swap where width should be the wall thickness (~100mm) and depth the wall length. This is a known data issue (CurrentState.txt §2.3) and contributes to the door depth anomalies.

Doors (6 IfcDoor)

Element	Host Wall	Fraction	Width (mm)	Material
IfcDoor_1	WALL_common_SOUTH	0.5	900	Door_D1_Main
IfcDoor_2	WALL_bilik_utama_EAST	0.3	800	Door_D2_Internal
IfcDoor_3	WALL_bilik_2_WEST	0.3	800	Door_D2_Internal
IfcDoor_5	WALL_tandas_EAST	0.5	700	Door_D3_Bathroom

IfcDoor_6	WALL_bilik_mandi_EAST	0.5	700	Door_D3_Bathroom

Defect trace: All doors have depth_mm=100 (the wall fallback). No orientation column exists — this is the root cause of defect D2 (doors not rotated to face corridor). The main door (IfcDoor_1) at fraction 0.5 on the south wall of the common room collides with FE_9 (sofa) also at fraction 0.5 — defect D3.

Windows (11 IfcWindow)

All windows carry material_rgba = 180,220,255,160 (int-255 format), which the transparency pipeline converts to alpha 0.627 (=160/255) for glass rendering.

Element	Host Wall	Fraction	Width	Material	Transparency
IfcWindow_1	bilik_utama SOUTH	0.5	1200	Window_W1	0.6
IfcWindow_2	common SOUTH	0.25	1800	Window_W2	0.6
IfcWindow_3	common SOUTH	0.75	1800	Window_W2	0.6
IfcWindow_7	bilik_mandi NORTH	0.5	600	Window_W3_Small	0.6
IfcWindow_8	bilik_utama WEST	0.5	100/1200*	Window_W1	0.6

**Width/depth swap:* Windows 8–11 on east/west walls show width_mm=100 and depth_mm=1200 — the dimensions are transposed relative to the wall plane. This is defect D7.

Furniture (10 IfcFurnishingElement)

Elemen	Host Room	Fractio	Width × Depth	Material

t		n	(mm)	
FE_1	common	0.3	1200 × 600	Kitchen_Sink_Aluminium
FE_2	bilik_mandi	0.5	500 × 400	Basin
FE_3	bilik_mandi	0.5	400 × 700	WC_Seating
FE_4	bilik_mandi	0.2	900 × 900	Shower
FE_5	tandas	0.5	400 × 700	WC_Seating
FE_6	bilik_utama	0.5	1500 × 1900	Bed_Wood
FE_7	bilik_2	0.5	900 × 1900	Bed_Wood
FE_8	bilik_3	0.5	900 × 1900	Bed_Wood
FE_9	common	0.5	2000 × 800	Sofa_Fabric
FE_10	common	0.5	1200 × 600	Coffee_Table_Wood

Defect traces: FE_5 (WC_Seating in tandas) has no rotation_deg → defect D4. FE_9 (Sofa at fraction 0.5 in common) collides with IfcDoor_1 (also fraction 0.5 on south wall) → defect D3. Only FE_1 serves the kitchen zone; no stove, fridge, or counter → defect D5. Only FE_9 + FE_10 in the 22.94 m² living area → defect D6.

B.6 Wall Type System

Wall thickness is determined by ad_wall_type, not hardcoded. The current table serves three construction traditions:

Wall Type ID	Thickness (mm)	Construction Tradition	Description
EXTERIOR_BRICK	417	US (Duplex)	Brick on block
INTERIOR_PARTITION_92	124	US (Duplex)	92mm stud + 2×16mm gypsum
PARTY_CMU	493	US (Duplex)	CMU demising wall
EXTERIOR_UK_BRICK_290	290	UK (SampleHouse)	102Bwk+75Ins+100LBlk+12P

INTERIOR_UK_PARTITION	95	UK (SampleHouse)	12P+70MStd+12P
EXTERIOR_MY_BRICK_150	150	MY (Terminal)	Brick plaster both sides
EXTERIOR_MY_BRICK_250	250	MY (Terminal)	Thick exterior
EXTERIOR_V1	MISSING	MY (CitizenHome)	Referenced but not defined — DATA GAP
INTERIOR_V2	MISSING	MY (CitizenHome)	Referenced but not defined — DATA GAP

The shaded rows show the data gap: CitizenHome references wall types that do not exist in the table, causing the compiler to fall back to 100mm for all openings. This is a two-row SQL fix, not a code change.

B.7 Transparency Pipeline

Material transparency flows through the system without modification by any Java code:

```
ad_element_rule.material_name   (e.g. 'Window_W1')
       ↓  [Java: RelationalResolver]
PlacementAD.Placement.materialName()
       ↓  [Java: BuildingWriter]
output DB: elements_meta.material_name
       ↓  [SQL: LEFT JOIN]
output DB: surface_styles.transparency  (e.g. 0.6)
       ↓  [Python: Bonsai Federation]
Blender: blend_method='BLEND', Alpha=0.4
```

The Java compiler acts as a pass-through for material data. Transparency values are authored in surface_styles and consumed by the Python exporter. No interpolation, clamping, or default substitution occurs in the resolver.

B.8 Compilation Mode: Relational vs. Flat

The compiler supports two compilation modes toggled by a single configuration value:

Property	FLAT Mode	RELATIONAL Mode
Data source	ad_element_placement (pre-computed XYZ)	ad_element_rule (grid + room + wall rules)
Coordinates	Stored as minX/maxX/minY/maxY/minZ/maxZ	Computed at runtime by RelationalResolver
SH rows	55	55 rules
DX rows	1085	1085 rules
Terminal rows	51,088	Not yet populated (FLAT only)
CitizenHome rows	None (generative)	58 rules
Validation	Serves as test oracle	Shadow-validated: 100% match (0.001mm error)

CitizenHome has no flat placement data. It can only compile in RELATIONAL mode. This is the formal proof that the system is generative: the compiler computes all coordinates from rules, not from extracted reference positions.

B.9 Summary: Table-to-Output Traceability

Every element's position, dimensions, material, and transparency is driven by one of these tables:

Table	What It Drives	Row Count
ad_compiler_config	Runtime mode switches	3
ad_building_grid	Grid coordinates (spatial skeleton)	≈70
ad_room_boundary	Room extents from grid references	≈50
ad_wall_face	Wall types, adjacency, exterior flag	≈120
ad_element_rule	Element placement rules	1,198

	(RELATIONAL)	
ad_element_placement	Flat pre-computed placement (FLAT)	68,472
ad_wall_type	Wall construction thicknesses	15
ad_geometry_map	Element → mesh mapping + provenance	≈200
surface_styles	Material PBR properties + transparency	80
ad_bom / ad_bom_child	Furniture/fixture assembly rules	22 / 82
ad_bom_child_param	Spatial offsets within assemblies	214

The architectural claim is: where output looks wrong (opening depth, door rotation, sparse furniture), the fix is always a data update — never a code change. Figures 1–4 and Tables 2–3 provide the evidence: every visible defect traces to a missing or incorrect value in a specific table cell.

Appendix C. Glossary

For readers encountering these terms for the first time.

BIM and IFC Terms

BIM (Building Information Modelling) A process for creating and managing digital representations of physical and functional characteristics of buildings. BIM models contain geometry plus semantic data (material, cost, schedule, manufacturer).

IFC (Industry Foundation Classes) An open, ISO-standardised data schema (ISO 16739) for describing building and construction data. IFC files enable interoperability between different BIM software tools. Current versions: IFC4 (buildings), IFC4.3 (infrastructure).

IFC Class (e.g., IfcWall, IfcDoor, IfcSlab) A semantic category in the IFC schema. Each class defines what an element is (a wall, a door, a floor slab) and what properties it can carry. The compiler emits elements tagged with IFC classes.

LOD (Level of Development) A framework (BIMForum, 2023) describing the reliability and detail of BIM elements. LOD 100 = conceptual, LOD 200 = approximate geometry, LOD 300 = precise geometry, LOD 350 = construction-coordination, LOD 400 = fabrication-ready with exact dimensions and manufacturer data.

BCF (BIM Collaboration Format) A buildingSMART standard for communicating issues (clashes, design queries, change requests) between BIM tools via lightweight XML files. Used by Text2BIM for its feedback loop.

Compiler and Architecture Terms

DSL (Domain-Specific Language) A small programming language designed for a specific problem domain. In this system, the DSL expresses building intent (“a three-bedroom house with porch”) in 20–80 lines, rather than specifying geometry directly.

DAG (Directed Acyclic Graph) A computational pipeline where each stage depends on previous stages but has no circular dependencies. The compiler’s five stages (Parse → Resolve → Compile → Bind → Write) form a DAG.

Metadata-Driven An architectural pattern where system behaviour is controlled by data in configuration tables rather than by procedural code. Adding a new building type requires inserting rows into metadata tables, not writing new code.

Application Dictionary (AD) A set of database tables that store domain knowledge as queryable data. Adapted from the iDempiere ERP system's pattern for managing product configuration complexity. In this system, 55 AD tables store all construction knowledge.

BoundElement A Java record type whose constructor mathematically proves that an element's mesh geometry fits within its placement bounding box. If the proof fails (scale factor outside [0.3, 3.0] per axis), the element cannot be constructed. This is the compiler's primary correctness gate.

Proof-Carrying Type A data structure whose constructor enforces invariants — the act of creating the value constitutes a proof that the invariants hold. Adapted from proof-carrying code (Necula, 1997) to the BIM domain.

MeshBinder The compiler stage that unifies two previously separate pipelines (bounding box placement and tessellated mesh geometry) by creating BoundElement records. This is the "Bind" stage in the DAG.

FederatedModel DB A SQLite database schema that stores IFC building data in SQL-queryable form: tessellated geometry as vertex/face arrays, spatial data as bounding boxes, semantic data as IFC class tags. Contributed to the IfcOpenShell project. Serves as the prerequisite container for both extraction (Phase 1) and compilation (Phase 2).

Bonsai An open-source Blender addon from the IfcOpenShell project for working with IFC data. The system's federation addon extends Bonsai to render buildings from the FederatedModel DB, enabling viewport display of 51,000+ elements that would exceed memory limits if loaded directly as IFC.

Extraction (Phase 1) The process of parsing a source IFC file and decomposing it into the FederatedModel DB schema. Lossless at LOD 400. Produces ground truth for Rosetta Stone validation.

Compilation (Phase 2) The process of expanding DSL declarations into positioned IFC elements through the DAG pipeline. Reads from metadata tables. Produces new building output from intent.

Validation Terms

Rosetta Stone (in this system) A reference IFC file paired with a DSL description of the same building. Named by analogy to the historical artefact that enabled translation between Egyptian hieroglyphs and Greek — the reference building enables translation between IFC semantics and DSL vocabulary.

Spatial X-Ray An algorithm that compares axis-aligned bounding boxes between the reference IFC and the compiler's output, using 10mm bucketing and asymmetric tolerances. Produces recall and precision scores. "X-ray" because it sees through visual appearance to spatial structure.

SpatialDigest A SHA-256 hash computed over all element bounding boxes at 1mm precision. Serves as a deterministic fingerprint: if any element changes position or size by even 1mm, the digest changes. Used for regression testing.

PlacementProver An audit component that checks 14 mathematical properties of the compiled building across 5 tiers: per-element arithmetic, pairwise relationships, host-element containment, topological properties, and conservation laws (perimeter length, floor area).

Witness Concrete evidence attached to compiler output that proves a specific claim about building correctness. Adapted from the mathematical concept where a witness is a value that demonstrates an existential claim. The compiler generates witnesses; a separate verifier checks them.

Compound Enrichment The property that each new building compiled permanently enriches six system layers: vocabulary, component activation, solver robustness, witness coverage, profile library, and spatial pattern maturity. This creates diminishing marginal effort for subsequent buildings of similar type.

Construction Terms

BOM (Bill of Materials) A structured list of all components needed to construct an assembly. In ERP systems, a BOM defines parent-child relationships between products and their constituent parts. This system uses BOMs to define furniture assemblies and fixture groupings.

Bounding Box (AABB) An Axis-Aligned Bounding Box — the smallest rectangular box aligned to the X/Y/Z axes that fully encloses an element's geometry. Used as the spatial primitive for the X-ray comparison.

Curtain Wall A non-structural facade system of glass panels (IfcPlate) held by metal mullions (IfcMember). In the SampleHouse, the curtain wall consists of 6 glass plates and 20 aluminium mullions.

Opening (IfcDoor, IfcWindow) Elements placed within walls that create voids. The "thin dimension" of an opening (perpendicular to wall face) should match the host wall's construction thickness, a rule that has required 6 separate fixes across the project's history.

Sill Height The height from floor level to the bottom of a window frame. Stored in ad_opening_family.sill_mm. Typical values: 900mm for living areas, 1500mm for bathrooms (privacy).

Party Wall A shared wall between two dwelling units (e.g., the central wall in a duplex). Typically thicker than interior partitions for fire and sound rating. The Duplex uses CMU demising at 493mm.

This document is intended for peer review and community feedback. Corrections, counterexamples, and alternative perspectives are welcomed. The goal is accurate characterisation of the field, not advocacy for any single approach.